VEGOVA
ELEKTROTEHNIŠKO-RAČUNALNIŠKA
STROKOVNA ŠOLA IN GIMNAZIJA
LJUBLJANA

Prvoosebna 3D računalniška igra v pogonu Unity

Strokovno poročilo

Mentor: dr. Matic Urbanija

Avtor: Boštjan Skubic, R 4. A

Ljubljana, April 2025

Povzetek

Projekt predstavlja razvoj prvoosebne 3D igre v okolju Unity, kjer igralec poskuša pobegniti iz zapora z reševanjem ugank in izogibanjem robotskemu pazniku. Igralec uporablja mehanike teka, počepa, interakcije s predmeti in upravljanja z inventarjem, medtem ko robot sledi igralcu s pomočjo NavMesh navigacije in dinamično prilagodljivega svetlobnega sistema. Ključna tehnika je uporaba C# skript za implementacijo mehanik. Razvoj je vključeval faze načrtovanja, prototipiranja, optimizacije svetlobe ter gradnjo končne verzije z uporabo orodij Git, Visual Studio in Unity Asset Store. Projekt združuje programerske veščine z uporabniško izkušnjo in umetniškim izrazom.

Ključne besede: prvoosebna 3D igra, Unity, umetna inteligenca, svetlobni sistem, logične uganke, uporabniški vmesnik, C#, skriptiranje, razvoj.

Kazalo Vsebine

1. Uvod 5
2. Opis projekta 6
2.1 Ključne mehanike 6
2.1.1 Mehanike igralca 6
2.1.2 Mehanike robota 6
2.1.3 Mehanike predmetov 7
2.1.4 Mehanika uporabniškega vmesnika 7
2.2 Uporabljena orodja 7
2.2.1 Unity 2022 LTS 7
2.2.2 Visual Studio 2019 8
2.2.3 Git in GitHub 9
3. Postopek izdelave 10
3.1 Načrtovanje 10
3.2 Spoznavanje orodja Unity 11
3.3 Izdelava delujočega prototipa 11
3.3.1 Izdelava prvih prostorov 11
3.3.2 Razvoj ključnih mehanik 12
3.4 Druga faza izdelave 14
3.4.1 Razširitev igralnega prostora 14
3.4.2 Prva uganka ..... **Error! Bookmark not defined.**
3.4.3 Izdelava glavnega menija 17
3.5 Končna faza izdelave 18
3.5.1 Ključavnica s kodo in zvočni namigi 18
3.5.2 Dodajanje tekstur 19
3.5.3 Dodajanje zvočnih efektov 19
3.5.4 Dodelava svetlobnega sistema 19
3.5.5 Build igre 20
4. Tehnična izvedba 20
4.1 Skripte igralca (Player) 21
4.1.1 Character controller 21
4.2 Interakcija 25
4.2.1 Sestavni deli sistema na objektih 25
4.2.2 Sestavni deli sistema na igralcu 27
4.3 Skript robota 28
4.3.1 Umetna inteligenca robota 28
4.4 Močne in šibke luči 31
5. Zaključek 33
6. Literatura in viri 34

Kazalo slik

Slika: 1: Nasprotnik v igri - robot 6
Slika: 2: Posnetek zaslona v orodju Unity 8
Slika: 3: Posnetek zaslona v orodju Visual Studio 2019 9
Slika: 4: Posnetek zaslona v orodju GitHub Desktop 9
Slika: 5: Skica robota 10
Slika: 6: Skica igralnega prostora 10
Slika: 7: Prototip skladišča 12
Slika: 8: Prikaz mreže NavMesh 12
Slika: 9: Močna svetloba 13
Slika: 10: Šibka svetloba 13
Slika: 11: Pobiranje predmetov 14
Slika: 12: Slika EMP puške in naboja 14
Slika: 13: Končen obseg igralnega prostora 15
Slika: 14: Prostor z omaricami 15
Slika: 15: Prostor za videonadzor 16
Slika: 16: Uganka z vrati in stikali 16
Slika: 17: Izhod iz zapora 17
Slika: 18: Glavni meni 18
Slika: 19: Skladiščna vrata in številčna kljulavnica 18
Slika: 20: Stene z teksturami 19
Slika: 21: Prikaz zvočnih posnetkov 19
Slika: 22: Izgled igre po generiranju svetlobnih zapisov 20
Slika: 23: Datoteke ki sestavljajo build igre 20

1. Uvod

Za izdelek pri 4. predmetu poklicne mature sem se odločil za izdelavo videoigre, saj menim da razvoj igre združuje različna področja znanja računalništva in programiranja z drugimi veščinami, kot so logično razmišljanje, načrtovanje uporabniške izkušnje, umetniško izražanje in tehnična izvedba.

Igra je izdelana v tridimenzionalnem okolju, saj so mi kot igralcu takšne igre bolj všeč. Odločil sem se za eno-igralsko zasnovo, ker sem se želel osredotočiti predvsem na logiko igranja, vizualno podobo ter mehanike igre, namesto na kompleksno implementacijo več-igralske funkcionalnosti.

Igra temelji na temačnem, napetem vzdušju, kjer se igralec prebudi v zaporu brez spomina in mora z reševanjem ugank ter izogibanjem rdeči svetlobi, najti pot na prostost. Igralec pa v zaporu ni sam. V njem je tudi robotski paznik, ki začne igralca loviti če ga zazna v svojem snopu svetlobe.

2. Opis projekta

2.1 Ključne mehanike

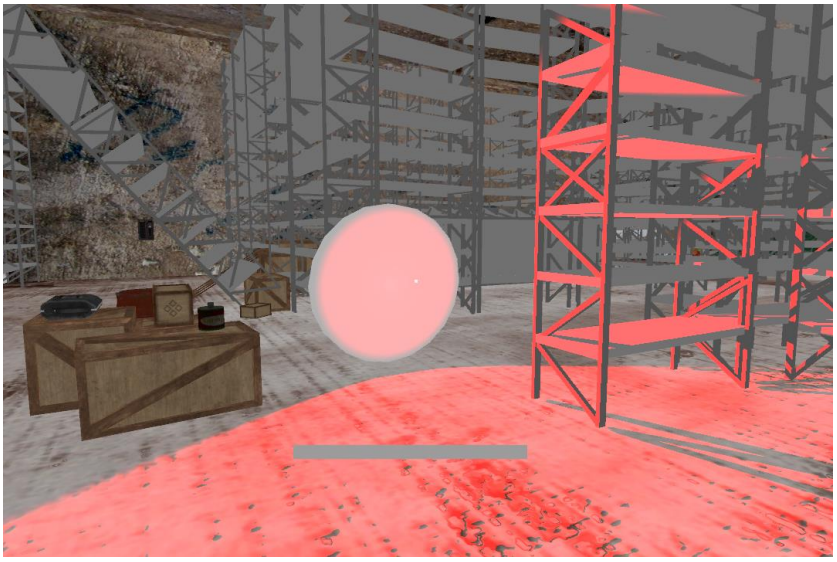
2.1.1 Mehanike igralca

Glavna mehanika, ki jo igralec potrebuje, je premikanje po prostoru. Ta mu bo omogočila raziskovanje in beg. Igralec ima možnost počepa, ki mu omogoči, da se skrije za višje predmete, a se premika počasneje in možnost teka, ki pa se mora ponovno generirati, ko igralec ne teče. Naslednja pomembna mehanika je pobiranje in hranjenje najdenih uporabnih predmetov v svojih rokah. Seveda mora igralec imeti tudi sposobnost uporabe teh predmetov.

2.1.2 Mehanike robota

Glavna mehanika robota je njegova umetna inteligenca. Robot se zna premikati po prostoru med ključnimi točkami. Ko robot v svojem snopu svetlobe zazna igralca, preneha premikanje med ključnimi vnaprej določenimi točkami in začne loviti igralca. Ko igralca ne vidi več, se zavrti okoli za 360 stopinj okoli sebe in poskuša igralca še zadnjič najti.

Ključen del, ki robotu doda bolj zanimivo vrednost kot nasprotniku, je njegov spremenljiv šop svetlobe. Ko samo patruljira med točkami, je njegov šop svetlobe srednje dolg in srednje širok. Ko igralca opazi, se njegov šop svetloba skrajša in postane širši. To mu pomaga pri sledenju igralcu. Ko igralca izgubi in naredi obrat okoli sebe, je njegov šop svetlobe zelo dolg in ozek.



Slika: 1: Nasprotnik v igri - robot

2.1.3 Mehanike predmetov

Pod mehanike predmetov spadajo vse mehanike, ki so potrebne za delovanje predmetov v okolju.

Glavna mehanika, ki deluje z igralčevo mehaniko interakcije, je ta ki določi kaj se bo zgodilo ko igralec uporabi določen predmet (npr. ko igralec interaktira z vrati, se morajo ta premakniti). Ta mehanika je prisotna povsod, kjer igralec uporablja katerekoli predmete v zaporu.

2.1.4 Mehanika uporabniškega vmesnika

Prva stvar, s katero se sreča igralec, ko zažene igro, je glavni meni. Na tem meniju ima igralec različne možnosti. Te so možnost začetka igre, možnost nastavitev in prikaza kontrol in izhod iz igre. V zavihku za nastavitve, so nastavitve zvoka, ki se morajo ohraniti tudi po tem ko je igralec začel igro.

Igralec ima ob igranju na zaslonu prikazano število nabojev za puško in koliko časa še lahko teče. Tako je stalno informiran o pomembnih podatkih. Ki jih potrebuje za beg pred robotom.

2.1.5 Uganke

Igralec mora za pobeg rešiti več ti. ugank. Igralec mora najti 3 velika stikala, ki ugasnejo smrtonosne luči pred izhodom. Eno izmed stikal se nahaja v prostoru z omaricami, kjer lahko igralec najde uporabno opremo. Da lahko pride do tega stikala mora najti pravilno kombinacijo manjših stikal v električni omarici, tako da bodo vsa vrata v prostor odprta hkrati. Drugi dve stikali se nahajata v skladišču, kjer se nahaja tudi robot. V kompleksu lahko najde 3 zvočne posnetke nekega prejšnjega zapornika, ki mu pomagajo ugotoviti številčno kombinacijo, ki odklene skladiščna vrata.

2.2 Uporabljena orodja

Za izdelavo izdelka sem uporabil različne računalniške programe, ki so mi neposredno pomagali pri izdelavi zaključenega projekta.

2.2.1 Unity 2022 LTS

Specifična različica: 2022.3.49f1

Glavno orodje, ki sem ga uporabil je zmogljivo in vsestransko okolje za razvoj iger (ang. game engine), ki omogoča izdelavo 2D in 3D aplikacij na različnih platformah, kot so Windows, Android, iOS, WebGL, konzole in druge. Zaradi svoje

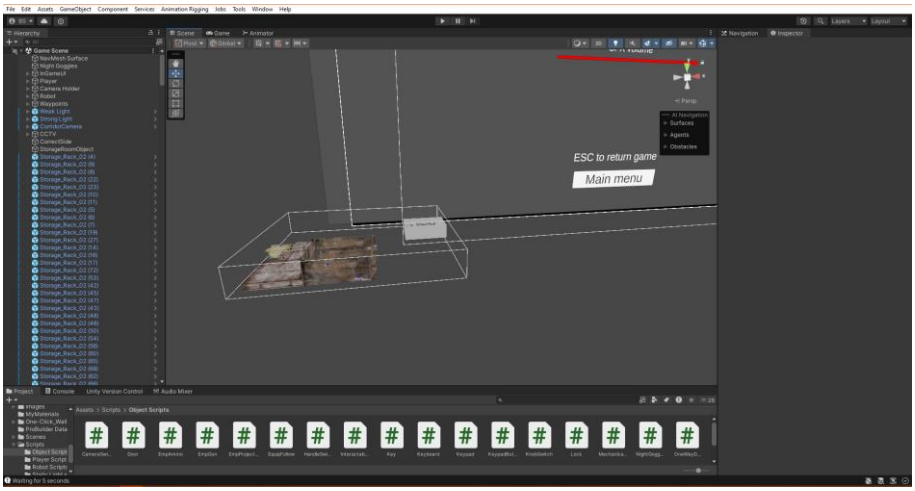
prilagodljivosti, široke podpore in obsežne dokumentacije je ena izmed najbolj priljubljenih izbir med neodvisnimi razvijalci iger in profesionalnimi studii.

Unity omogoča vizualno urejanje prizorišč preko svojega urejevalnika, kjer lahko razvijalec postavlja objekte, svetlobne vire, kamere, skripte in druge komponente. Poleg tega omogoča zelo močan fizikalni pogon, ki omogoča simulacijo gibanja, trkov, gravitacije in drugih pojavov.

Zelo pomembna lastnost Unityja je njegova podpra C# »skriptiranju«, kar razvijalcu omogoča popoln nadzor nad logiko igre in obnaša jem posameznih objektov.

Unity vsebuje tudi široko knjižnico brezplačnih in plačljivih sredstev (predvsem 3D modelov), kjer razvijalci med sabo delijo že pripravljene modele, skripte, zvočne efekte, animacije in druga sredstva, kar zelo pospeši razvoj videoigre. Vsa sredstva, za katere imamo licenco, lahko preprosto dodamo v naš projekt brez vmesnih korakov.

Odločil sem se za različico 2022 LTS (LTS – long term support), saj je že dolgo v uporabi in ima ogromno podpore in dokumentacije in je tudi zadnja LTS izdaja.

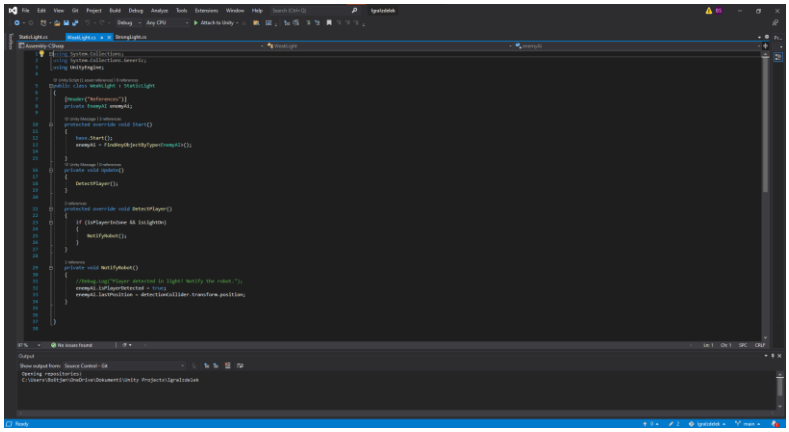


Slika: 2: Posnetek zaslona v orodju Unity

2.2.2 Visual Studio 2019

Visual studio je integrirano razvojno okolje (IDE), ki ga razvija Microsoft. Uporablja se za pisanje, urejanje, pregledovanje in razhroščevanje kode v različnih programskih jezikih. V pogonu Unity je Visual Studio nastavljen kot privzeti urejevalnik. Uporablja se ga za pisanje C# skript, ki se uporabljajo za implementacijo logike.

Visual Studio je tesno integriran z Unityjem, kar pomeni, da se spremembe v kodi takoj odražajo v pogonu, z napredno podporo za Unity-specifične funkcije, kot so MonoBehaviour razredi, Unity dogodki in serializacija objektov.



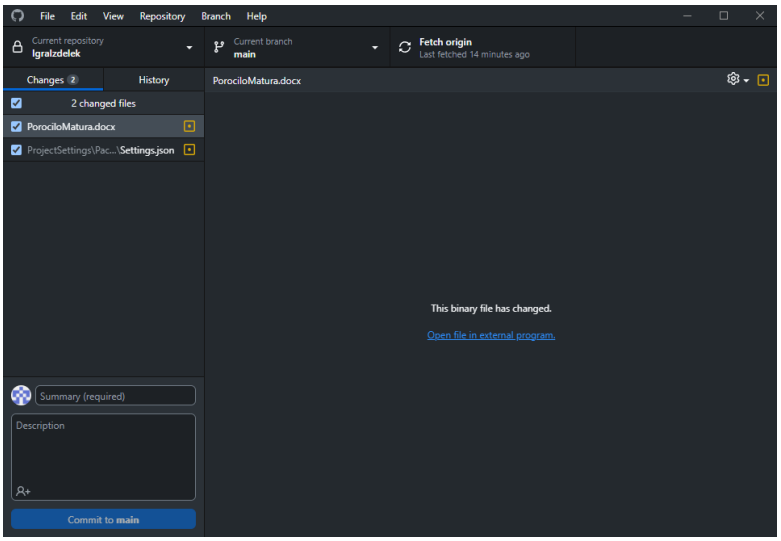
Slika: 3: Posnetek zaslona v orodju Visual Studio 2019

2.2.3 Git in GitHub

Git je sistem za nadzor različic, ki omogoča sledenje spremembam v izvorni kodi skozi čas. Omogoča razvijalcu, da shrani posamezne "posnetke" projekta (ang. commits), se po potrebi vrne na prejšnjo različico, in razvija različne funkcionalnosti ločeno preko *vej* (ang. *branches*). Git olajša delo na večjih projektih in preprečuje izgubo podatkov ob večjih spremembah.

GitHub pa je spletna platforma za gostovanje Git repozitorijev. Omogoča varno shranjevanje kode v oblaku, deljenje projektov z drugimi in enostavno sodelovanje v ekipah. Uporaba GitHuba omogoča tudi avtomatsko ustvarjanje varnostnih kopij in dostop do projekta iz katere koli naprave.

V mojem primeru sem Git uporabljal za sledenje različicam projekta, še posebej po tem, ko sem po nesreči izgubil celotno sceno in povezave v Unityju. GitHub pa mi je služil saj sem projekt razvijal na dveh napravah in sem tako vedno imel najnovejšo različico projekta na obeh napravah.



Slika: 4: Posnetek zaslona v orodju GitHub Desktop

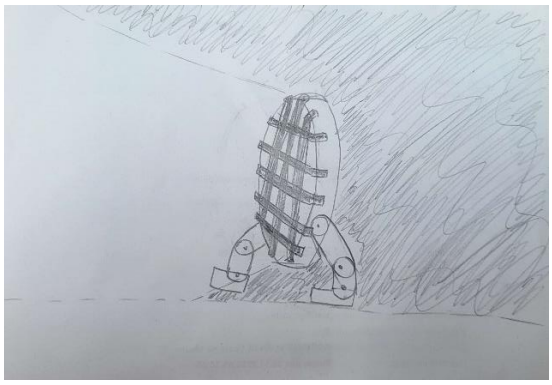
3. Postopek izdelave

Podrobneje bom opisal kako sem se lotil izdelave videoigre. Bolj se bom osredotočil na različne metode in ne toliko na dejanske dele igre. Na to se bom osredotočil v naslednjem poglavju.

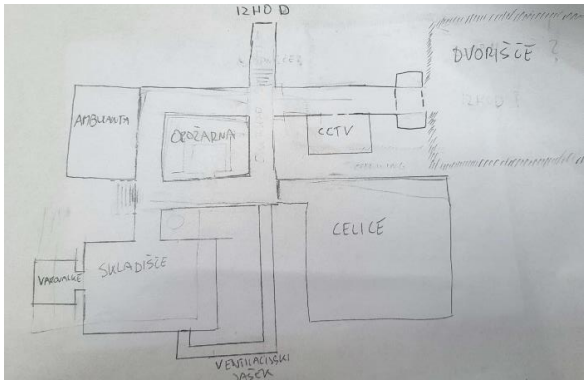
3.1 Načrtovanje

Na začetku sem si zastavil osnovno idejo igre in cilje, ki jih želim z njo doseči. S tem sem imel kar nekaj problemov, saj si nisem hotel zastaviti nečesa, česar nebom mogel izvesti. Idejo sem razvijal okoli ocene svojih zmožnosti in o svoji preferenci za igre.

Po kar dolgih premislekih sem končno pristal na končno idejo. Inspiracijo za igro sem vzel iz različnih prvoosebnih iger iz žanre »horror survival«. Mentorju sem oddal dokument, z natančnejšimi opisi funkcionalnost in ostalimi opisi igre. Naknadno sem narisal tudi načrt igralnega prostora, vendar se je kar zelo spremenil.



Slika: 5: Skica robota



Slika: 6: Skica igralnega prostora

3.2 Spoznavanje orodja Unity

Ker orodja Unity v takšni meri, kot sem ga rabil uporabljati za ta izdelek, še nisem uporabljal, sem se z njim seznanil preko videoposnetkov iz spletne platforme YouTube, z uporabo umetne inteligence in uporabo uradne dokumentacije. Ti viri so mi omogočili da sem se hitro naučil osnovnih principov delovanja orodja in uporabe skript v jeziku C# ter različnih Unityjevih razredov in metod. Z njihovo pomočjo sem lahko vzpostavil nekaj začetnih funkcionalnosti, ki so bile ključne za nadaljnji razvoj igre.

Najbolj pomembna naloga je bila implementacija premikanje igralca. To je predstavljalo osnovo za testiranje drugih mehanik, kot so interakcija s predmeti, zaznavanje svetlobe in delovanje sovražnika. Ko sem pridobil več znanja in boljše razumevanje principov objektno usmerjenega programiranja znotraj Unity-ja, sem vse začetne skripte preuredil ali jih povsem na novo napisal. Na ta način sem jih prilagodil svojemu načinu kodiranja in zagotovil, da so delovale usklajeno z ostalimi funkcionalnostmi igre.

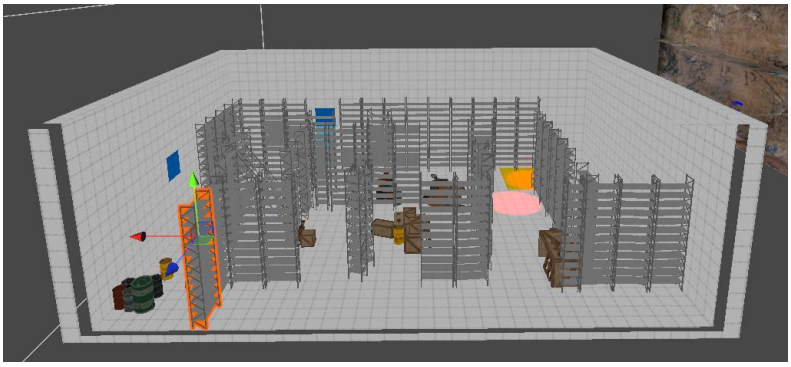
Ta proces sprotnega učenja je bil ključen, saj mi je omogočil, da sem svoje znanje poglobil in ga hkrati uporabljal.

3.3 Izdelava delujočega prototipa

Ko je bilo moje znanje bolj utrjeno, sem se lotil prototipa. Lotil sem se implementacije novih mehanik. Odločil sem se da bom hitro vpeljal čim več glavnih mehanik tako, da bom na koncu imel trdno ogrodje kateremu lahko kasneje dodam ostale manjše funkcionalnosti. V tej fazi dela sem tudi izgubil celotno glavno sceno in vse povezave med referencami in skripti. Tu sem projekt zavaroval z vodenjem različic in uporabo orodja Git.

3.3.1 Izdelava prvih prostorov

Najprej sem ustvaril osrednji prostor – skladišče, kjer se odvija večina interakcij med igralcem in robotom. Nato sem dodal še sobo za videonadzor, ki igralcu omogoča opazovanje drugih prostorov. Prostore sem oblikoval s pomočjo orodja ProBuilder, 3D modele pa sem pridobil iz Asset Store-a. Skladišče je zasnovano tako, da omogoča skrivanje in različne poti, medtem ko je soba za nadzor varna cona z mrtvim paznikom, ki drži ključ.



Slika: 7: Prototip skladišča

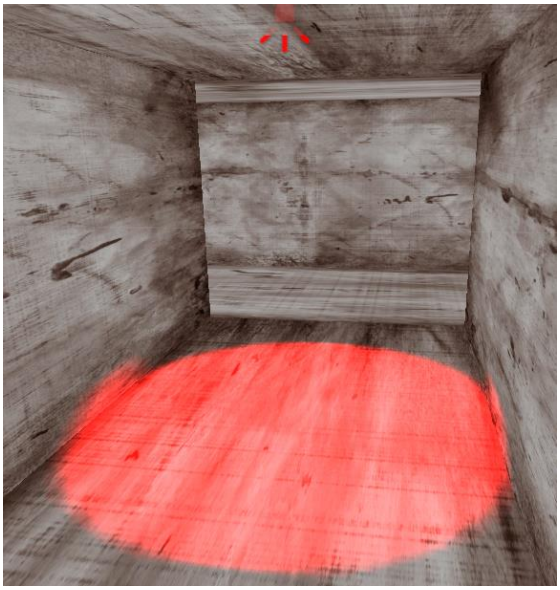
3.3.2 Razvoj ključnih mehanik

Umetna inteligenca robota temelji na orodju **NavMesh**, ki omogoča premikanje robota med točkami in reagiranje na prisotnost igralca. Ko ga robot opazi, zasleduje njegovo zadnjo znano lokacijo, nato se vrne na patruljno pot.



Slika: 8: Prikaz mreže NavMesh

Sistem svetlobe vključuje šibke in močne svetlobne snope. Šibka svetloba razkrije igralčevo lokacijo, močna pa povzroči smrt (kar se trenutno izrazi le z obvestilom na zaslonu).

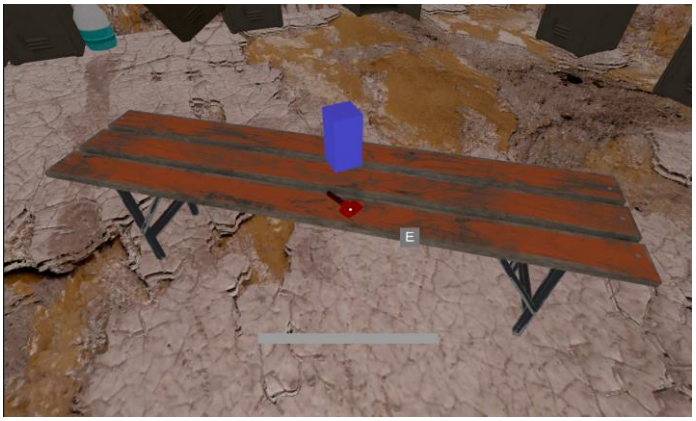


Slika: 9: Močna svetloba



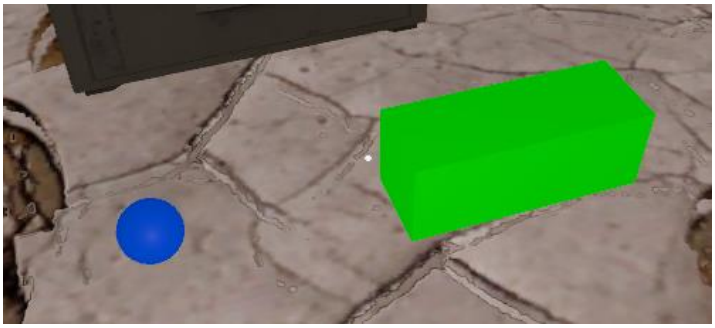
Slika: 10: Šibka svetloba

Interakcijski sistem omogoča pobiranje predmetov, dvomestni inventar, uporabo ključev za odklepanje ter uporabo orožja.



Slika: 11: Pobiranje predmetov

Orožje in nočna očala: Igralec lahko najde pištolo za začasno onesposobitev robota in nočna očala, ki povečajo vidljivost v temi. Očala ne zavzamejo prostora v inventarju in ostanejo aktivna od trenutka, ko jih igralec pobere.



Slika: 12: Slika EMP puške in naboja

3.4 Druga faza izdelave

Po uspešni predstavitvi prototipa na drugi konzultaciji, sem nadaljeval z izdelavo igre. Osredotočil sem se na širjenje igralnega prostora, uvajanje prvih ugank in izboljšanje igralne izkušnje z dodatnimi vizualnimi in funkcionalnimi elementi. V tej fazi sem tudi začel urejati uporabniški vmesnik in pripravil glavni meni.

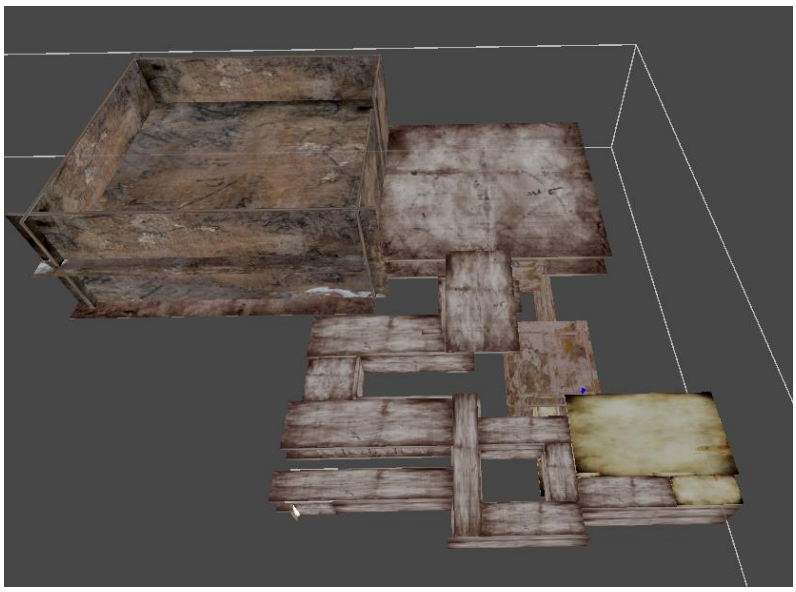
3.4.1 Razširitev igralnega prostora

Dodal sem več novih prostorov in jih med seboj povezal s hodniki, pri čemer sem še vedno uporabljal orodje ProBuilder. Po mentorjevi povratni informaciji sem znatno povečal skladiščno sobo (približno trikrat), kar omogoča več poti za gibanje in skrivanje ter s tem več taktičnih možnosti za igralca.

Med pomembnejšimi novimi prostori je tudi hodnik s celicami, ki služi kot začetna točka igre. V celice sem postavil 3D modele postelj in zapornikov, nekatere sem pustil

prazne. V tiste, ki imajo zapornika, sem postavil močno rdečo luč, ki z igralčeve perspektive “paralizira” zapornike.

Dodana je bila tudi soba z osebnimi omaricami paznikov, v kateri igralec najde uporabne predmete. Dve omarici sta zaklenjeni – za odpiranje je potrebno najprej odkleniti ključavnico. V eni se nahaja puška, v drugi pa nočna očala.



Slika: 13: Končen obseg igralnega prostora



Slika: 14: Prostor z omaricami



Slika: 15: Prostor za videonadzor

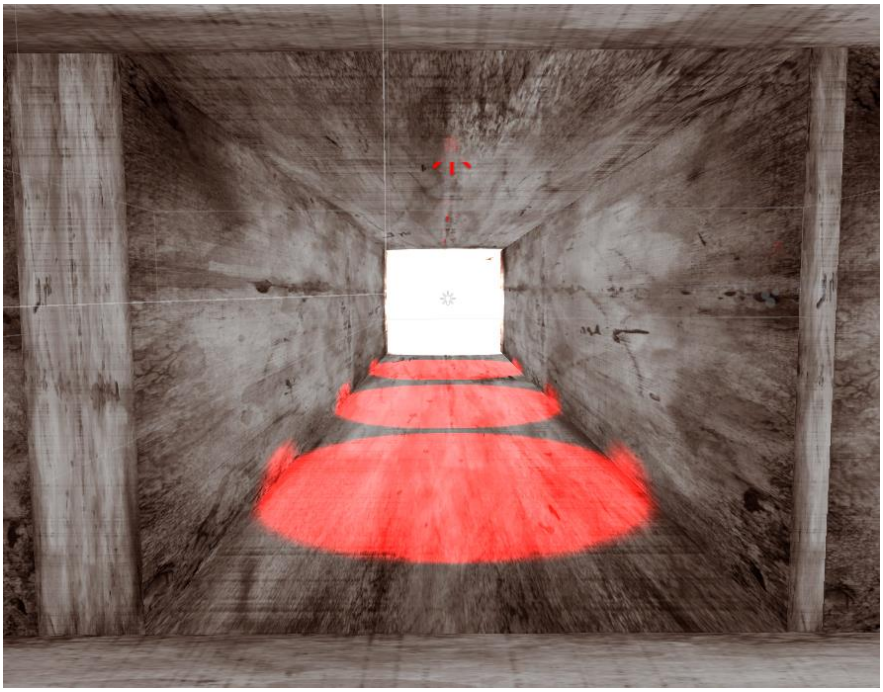
3.4.2 Uganke

Pred sobo z omaricami sem postavil prvo logično uganko. Gre za prostor s petimi zaporednimi vrati, ki so povezana s stikali. Vsako stikalo vpliva na določena vrata (odpre jih ali zapre), cilj pa je, da igralec s pravilnim zaporedjem aktivacij pride do naslednjega prostora. Ta sistem sem sam implementiral s pomočjo skript, ki definirajo povezave med stikali in vrati.



Slika: 16: Uganka z vrati in stikali

Nato sem v po igralnem prostoru razpršil še glavna stikala, ki ugasnejo luči pred izhodom.



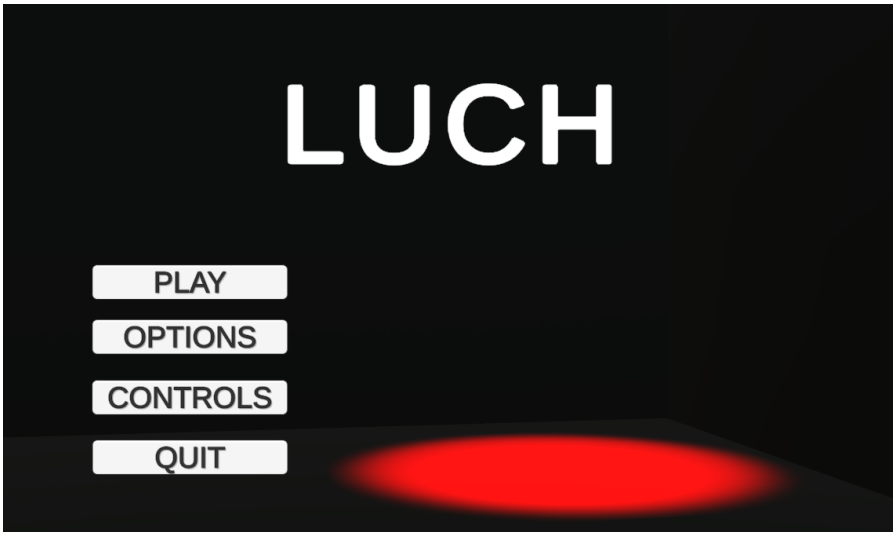
Slika: 17: Izhod iz zapora

3.4.3 Izdelava glavnega menija

Začetni meni sem izdelal v ločeni sceni. Igralcu omogoča pregled nad kontrolami, spremembo zvočnih nastavitev in začetek igre. Namesto klasičnega 2D ozadja sem ustvaril temno 3D okolje z utripajočo lučjo, ki že uvaja vzdušje igre. Barvna shema menija sledi tonu igre – uporablja temne, sive in rdeče odtenke.

Dodana sta bila dva podmenija:

- “Nastavitve”, kjer igralec nastavi glasnost glasbe in učinkov,
- “Kontrole”, kjer so prikazane vse tipke, ki jih igralec potrebuje med igro.



Slika: 18: Glavni meni

3.5 Končna faza izdelave

V zaključni fazi sem se osredotočil na izboljšanje igralne izkušnje, odpravo napak, dodajanje manjkajočih elementov izgled igre ter optimizacijo in pripravo igre na predstavitev.

3.5.1 Ključavnica s kodo in zvočni namigi

V tej fazi sem ob skladiščna vrata dodal številčnico (keypad), ki omogoča vnos štirimestne kode za odklepanje glavnih vrat. Da bi igralca vodil do rešitve, sem vključil zvočne posnetke, kjer bivši zapornik, ki je iz zapora zbežal, igralcu daje namige kako ugotoviti pravilno kodo za ključavnico. Tako igralec pridobi občutek vodenja, brez da bi bil proces prelahek ali preveč očiten.



Slika: 19: Skladiščna vrata in številčna kljulavnica

3.5.2 Dodajanje tekstur

Za bolj realistično in privlačno izkušnjo sem površinam dodal teksture, tako da so imele dejanski izgled. Poleg tega sem v okolje vključil dodatne 3D modele, s čimer sem izboljšal ubčutek zapuščenega in nevarnega prostora.



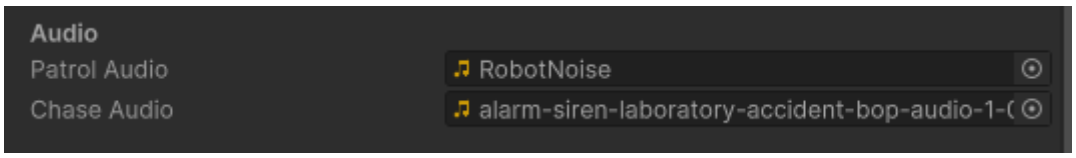
Slika: 20: Stene z teksturami

3.5.3 Dodajanje zvočnih efektov

Da bi igralec lažje razumel, v kateri fazi je robot (ali igralca vidi in ga lovi ali ne), sem mu dodal dva zvočna efekta:

- Ko mirno patruljira, spušča stalen miren zvok statike
- Ko igralca vidi in ga lovi spušča nevaren zvok alarma

S tem sem omogočil boljšo odzivnost igralca in poboljšal igralsko izkušnjo



Slika: 21: Prikaz zvočnih posnetkov

3.5.4 Dodelava svetlobnega sistema

Razliko med šibkimi in močnimi snopi svetlobe sem naredil bolj očitno. Svetlobo sem tudi popekel (ang. Bake), tako da je igra Tekla bolj tekoče, saj je bilo potrebnih manj kalkulacij, kar je pripomoglo k bolj tekoči igri in igralski izkušnji. Igra je začela pridobivati lep realističen izgled.



Slika: 22: Izgled igre po generiranju svetlobnih zapisov

3.5.5 Build igre

Kot zadnji korak sem pripravil prvi build igre, ki omogoča zagon igre zunaj okolja Unity. V buildu sem testiral, če prihaja do kašnih napak in če je igra tekoča. Igro sem tudi dal na preizkus sošolcem, saj sem tako najlažje dobil povratno informacijo in ugotovitl, kje so še kašne napake in kaj bi lahko izboljša.

Luch_BurstDebugInformation_DoNotShip	11/04/2025 06:25	File folder	
Luch_Data	11/04/2025 06:25	File folder	
MonoBleedingEdge	11/04/2025 06:25	File folder	
Luch.exe	11/04/2025 06:25	Application	651 KB
UnityCrashHandler64.exe	11/04/2025 06:25	Application	1,158 KB
UnityPlayer.dll	11/04/2025 06:25	Application exten...	30,107 KB

Slika: 23: Datoteke ki sestavljajo build igre

4. Tehnična izvedba

V tem poglavju bom po sklopih podrobneje opisal sestavne dele moje igre. Razložil bom pomembne skripte in konfiguracije objektov v sceni. Znotraj posameznih skript bom razložil le ključne metode. Izogibal se bom razlaganju spremenljivk, in jih v predstavljenih skriptah nebom definiral, kakor bi bilo potrebno za delovanje skripte.

Da bo dokument bolje strukturiran in bolj kompakten, bom na hitro predstavil osnovno obliko C# skript v Unityju. Če sem obliko kakšne skripte prilagodil ali dodal kakšne druge knjižnice pa bom to primerno dopisal.

Ko ustvarimo novo C# skripto Unityju, dobimo osnovno predlogo, ki izgleda takole:

```
using UnityEngine;
```

```
public class NewBehaviourScript : MonoBehaviour
{
    void Start()
    {

    }

    void Update()
    {

    }
}
```

Ta predloga vsebuje nekaj osnovnih delov:

- *Using UnityEngine → uporabi glavno Unity knjižnico*, ki vključuje vse Unity komponente kot so Monobehaviour, GameObject, Transform, Vector3 in druge, ki so nujno za delo znotraj Unity okolja
- *Public class NewBehaviourScript : MonoBehaviour* → definira novo javno C# klaso, ki podeduje MonoBehaviour. To pomeni, da lahko skripta deluje kot komponenta, ki jo lahko dodajamo objektom v Unityju.
- *Void Start()* → je metoda, ki jo Unity samodejno pokliče, ko se objekt prvič aktivira v igri. Pogosto jo uporabljamo za inicializacijo spremenljivk ali nastavitev začetnega stanja igre
- Void Update() → *je metoda, ki jo Unity kliče vsako sličico (ang. frame). V tej metodi se običajno nahaja logika, ki se mora neprestano preverjati ali izvajati.*

4.1 Skripte igralca (Player)

Skripte igralca so vse skripte, ki so postavljene na objekt igralca v glavni sceni igre. Te skripte skrbijo za premikanje in inventar. Na igralcu je tudi nekaj skript, ki so uporabljene za sistem interakcije, vendar jih bom zaradi lažje razumljivosti in organiziranosti opisal v ločenem poglavju.

4.1.1 Character controller

Razvijalci video iger uporabljajo izraz character controller za poimenovanje skripte, ki omogoči igralcu, da se premika, teče, skoči, počepne in druge stvari. Tudi sam sem s pomočjo že zelo dobrih obstoječih skript napisal svoj character controller, ki je bil najbolj primeren za mojo igro. Odločil sem se za uporabo fizikalnega kontrolerja, kjer igralca premikajo sile. Tako se bodo lažji predmeti, s katerimi igralec trči, premaknili oziroma se prevrnili, kar doda bolj realističen učinek.

```
private void GetInput()
{
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");
}
```

Metodo GetInput v tej skripti uporabljam za pridobitev horizontalnega in vertikalnega premika miške, ki je pomemben za rotiranje kamere glede na igralčev premik miške.

```
private void MovePlayer()
{
    moveDirection = Orientation.forward * verticalInput + Orientation.right *
horizontalInput;

     if (isGrounded)
    {
           rb.drag = groundDrag;
         rb.AddForce(moveDirection.normalized * moveSpeed * 10f,
ForceMode.Force);
     }
     else
     {
           rb.drag = 0;
         rb.AddForce(moveDirection.normalized * moveSpeed * airMultiplier * 10f,
ForceMode.Force);
    }
}
```

Metoda MovePlayer, združi premike miške in dobi vektor, ki določa smer v katero naj se igralec premakne. Preveri tudi če je igralec na tleh. To je zelo pomembno pri uporabi fizikalnega kontrolerja, saj mora v premiku upoštevati tudi trenje tal, ali nasprotno ko je igrelec v zraku. Nato na igralca doda silo, ki ga premakne v pravilni smeri.

```
private void RotatePlayer()
{
    Vector3 cameraRotation = Camera.main.transform.eulerAngles;
     transform.rotation = Quaternion.Euler(0, cameraRotation.y, 0);
}
```

Metoda RotatePlayer je pomembna za to, da se orientacija fizičnega modela igralca orientira skupaj s kamero, vendar le po y osi.

```
private void HandleSpeed()
{
    if (staminaController.CanSprint && Input.GetKey(KeyCode.LeftShift) &&
crouchController.IsCrouching)
    {
           moveSpeed = Mathf.Lerp(moveSpeed, crouchSprintMoveSpeed, sprintTransitionSpeed
* Time.deltaTime);
    }

    else if (staminaController.CanSprint && Input.GetKey(KeyCode.LeftShift))
    {
           moveSpeed = Mathf.Lerp(moveSpeed, sprintMoveSpeed, sprintTransitionSpeed *
Time.deltaTime);
    }

    else if (crouchController.IsCrouching)
    {
           moveSpeed = Mathf.Lerp(moveSpeed, crouchMoveSpeed, crouchController.CrouchSpeed
* Time.deltaTime);
    }

    else
    {
           moveSpeed = Mathf.Lerp(moveSpeed, walkMoveSpeed, sprintTransitionSpeed *
Time.deltaTime);
    }
}
```

Metoda HandleSpeed ima nalogo nastavljanja spremenljivke moveSpeed. Preveri če igralec hodi, šprinta ali čepi in tako spremenljivko moveSpeed nastavi na aktualno hitrost ki je nato uporabljena v že omenjeni metodi MovePlayer

4.1.2 Inventar

Sistem inventarja igralcu omogoča hranjenje dveh predmetov. Men njima lahko menja s pritiskom na gumb 1 ali 2. Predmet pobere s pomočjo sistema interakcije s pritiskom tipke E. Predmet lahko tudi spusti z pritiskom na tipko Q.

```
public GameObject[] inventorySlots = new GameObject[2];
```

Osrednji del funkcionalnosti inventarja je niz dveh GameObjectov (to so vsi predmeti, ki jih je možno postavljati na sceno v urejevalniku). Niz ima dva indeksa kar da ima igralec lahko hkrati na sebi dva predmeta.

```
private void EquipItem(int slotIndex)
{

    if (currentItem != null)
     {
           currentItem.SetActive(false);
         inventorySlots[currentIndex] = currentItem; // Save the item
     }

     currentItem = inventorySlots[slotIndex];
     currentIndex = slotIndex;

     currentItem.SetActive(true);
     SetInHand(currentItem);

     pickableItem = currentItem.GetComponent<PickableItem>();
     if(pickableItem != null)
     {
           pickableItem.SetItemHeld(true);
     }
}
```

Naloga metode EquipItem je, da pogleda če se na indeksu inventarja nahaja predmet. V primeru da se, kliče metodo SetInHand, ki bo izvedla dejansko vizualno spremembo predmeta v roki.

```
private void SetInHand(GameObject item)
{
    currentItem = item;

     EquipFollow followScript = currentItem.GetComponent<EquipFollow>();
     if (followScript == null)
     {
           followScript = currentItem.AddComponent<EquipFollow>();
     }
     followScript.equipPos = equipPos;
     currentItem.GetComponent<Rigidbody>().isKinematic = true;
     currentItem.GetComponent<Collider>().enabled = false;
     isHolding = true;
}
```

Metoda SetInHand dobi kot parameter objekt, ki naj ga postavi v roko. Na predmetu omogoči komponento EquipFollow, ki je skripta ki skrbi za to, da se predmet premka skupaj z igralcem. Komponento Rigidbody (skrbi za delovanje gravitacije in drugih delovanje sil na predmet) na predmetu sem nastavil na isKinematic, kar pomeni da nanj sile, ko je v roki ne delujejo.

```
private void HandleSlotSwitch()
{
    if (Input.GetKeyDown(KeyCode.Alpha1) && currentIndex != 0 &&
inventorySlots[0] != null)
     {
           EquipItem(0);
     }
     else if (Input.GetKeyDown(KeyCode.Alpha2) && currentIndex != 1 &&
inventorySlots[1] != null)
     {
           EquipItem(1);
     }
}
```

Metoda HandleSlotSwitch, preveri če je bil pritisnjen gumb 1 ali gumb 2 na tipkovnici. Če se je to zgodilo, kliče metodo EquipItem s primernim indeksom niza inventarja.

```
public void AddToInventory(GameObject item)
{
    if (TryAddItemToSlot(currentIndex, item) ||
    TryAddItemToSlot(GetOtherSlotIndex(), item))
           {
           return;
           }
}
```

Metoda AddToInventory je metoda, ki sama poskrbi da je predmet dodan na prazno mesto v inventarju, tudi če smo obrali predmet s polno roko.

Del te skripte je še nekaj drugih metod, ki pa so klicane iz skript interakcijskega sistema, zato jih bom opisal v naslednjem sklopu

4.2 Interakcija

Logika za interakcijo igralca z ostalimi predmeti v glavni sceni igre, je eden glavnih delov igralca. Omogoča mu:

- Odpiranje/zapiranje vrat
- Pobiranje predmetov
- Pritisk gumbov
- Uporabo predmetov

Sistem interakcije je ločen na več skript:

- Skripte z metodo Interact, kjer je definirano, kaj naj se zgodi ko igralec s tem predmetom interaktira
- Skripta za detekcija predmetov za interakcijo
- Skripta za klicanje Interact metode na predmetu, ki je bil uporabljen

4.2.1 Sestavni deli sistema na objektih

Skripte na interaktivnih predmetih, ki določajo kaj naj se zgodi, ko igralec izvede interakcijo (pritisk tipke E). Tukaj sem se sam prvič srečal z objektno orientiranim programiranje. Moral sem biti previden pri uporabi pravilnih modifikatorjih dostopnosti in kako razrede dedovati.

Ogrodje

```
public class Interactable : MonoBehaviour
{
    public virtual void Interact(){}
}
```

To je osnovni razred, na katerem stoji celoten sistem interakcije. Deduje iz razreda MonoBehaviour, tako da so za uporabo na voljo vse metode Unityja.

```
public class PickableItem : Interactable
{
    protected InventoryHandler inventory;
    public Transform playerCamera;
    public bool isHeld = false;

    public string rotationType;

     protected virtual void Start()
     {
        playerCamera = GameObject.Find("Camera Holder").transform;
        inventory = FindObjectOfType<InventoryHandler>();
     }

    public override void Interact()
    {
        inventory.AddToInventory(gameObject);
    }

    public void SetItemHeld(bool isHeldStatus)
    {
        isHeld = isHeldStatus;
    }
}
```

Razred PickableItem, deduje iz razreda Interactable. Najbolj pomembna je metoda Interact. Metoda Interact se izvede ko igralec na predmetu pritisne tipko E (skripta za to bo razložena v nadaljevanju). Znotraj metode Interact, je klic metode AddToInventory iz skripte za inventar, ki sem jo že opisal.

```
public abstract class UsableItem : PickableItem
{
    public abstract void Use();
}
```

V razred UsableItem spadajo vsi predmeti, ki so lahko pobrani in imajo neko uporabno vrednost (ključi, audio posnetki). Razred deduje iz prej opisanega razreda PickableItem, saj so vsi uporabni predmeti lahko tudi pobrani (a ne obratno). Glavna metoda, ki jo mora imeti vsak predmet, ki je lahko pobran, je metoda Use.

4.2.2 Sestavni deli sistema na igralcu

Sistem interakcije na igralcu je sestavljen iz dveh skript. Naloga prve je zaznavanje predmetov s katerimi lahko igralec izvede interakcijo. Naloga druge je klic pravilne metoda na skripti predmeta, ko z njim izvedemo interakcijo.

```
public class DetectionHandler : MonoBehaviour
{
    public Interactable GetDetectedInteractable()
    {
        RaycastHit hit;
        Ray ray = playerCamera.ScreenPointToRay(Input.mousePosition);

        if (Physics.Raycast(ray, out hit, interactionRange, interactableLayer))
        {
            if(hit.transform.GetComponent<Interactable>() != null)
            {
                detectedInteractable =
hit.transform.GetComponent<Interactable>();
                return detectedInteractable;
            }
        }

        detectedInteractable = null;
        return null;
    }

    public bool IsInteractableDetected()
    {
        return detectedInteractable != null;
    }
}
```

Znotraj skripte DetectionHandler je najbolj pomembna funkcija GetDetectedInteractable, ki je zadolžena za zaznavanje interaktivnih predmetov. Zazna jih s pomočjo žarka (raycast), ki je izstreljen iz sredine zaslona. Zelo pomembno je, da sem mu določil, da vrača le predmete, ki so del plasti interactable. Plasti so atribut vseh objektov v Unity okolju. S pomočjo plasti jih lahko med seboj logično grupiramo.

```
public class PlayerInteraction : MonoBehaviour
{
    private void Start()
    {
        pressE.enabled = false;
    }

    void Update()
    {
        if(interactable != null)
        {
            pressE.enabled = true;

            if (Input.GetKeyDown(KeyCode.E))
            {
                interactable = detectionHandler.GetDetectedInteractable()
                interactable.Interact();
            }
        }

        else
        {
            pressE.enabled = false;
        }
    }
}
```

Skripta PlayerInteraction je zadolžena za klicanje funkcij po temu, ko igralec pritisno tipko E na interaktivnem predmetu. Skripta dobi komponento predmeta, s katerim hočemo izvesti interakcijo in jo nato kliče.

4.3 Skript robota

Osrednji del robota, so njegova umetna inteligenca in preklapljanje med stanjema patruljiranja in lovljenja. Vso logiko robota imam združeno v eni skripti.

4.3.1 Umetna inteligenca robota

```
using UnityEngine.AI;
```

V skripti ki skrbi za obnašanje robota, je poleg knjižnjice UnityEngine, uporabljena tudi knjižnica UnitiEngine.AI, ki nam omogoča uporabo orodja NavMesh (navigacijska mreža). To je vgrajeno orodje, ki omogoča agentu (v mojem primeru je to robot) dinamično iskanje poti in premikanje po njih.

NavMesh temelji na 3D mreži, ki jo Unity generira glede na obliko terena in objektov na sceni. Ta mreža predstavlja površine, po katerih se agent lahko premika.

Nav mesh sem uporabil na robotu, da se lahko samostojno premika med točkami in zasleduje igralca. Agent samodejno izračuna pot okoli ovir (sten, škatel itd.), kar omogoča bolj naravno gibanje in manj ročnega programiranja.

```
using System.Collections;
```

Z vključitvijo knjižnice System.Collections, dobimo dostop do nekaterih drugih razredov in metod. Za to skripto je pomemben razred IEnumerator. Ta v Unityju omogoča izvajanje korutin – posebnih funkcij, ki potekajo skozi čas, namesto da se izvedejo v enem koraku. Uporablja se skupaj z yield in return, da lahko ustavimo izvajanje in ga nadaljujemo kasneje (npr. po določenem času ali pogojih), kar je idealno za zamike, animacije ali čakanje.

```
void Patrol()
{
    agent.speed = patrolSpeed;
    agent.destination = waypoints[targetWaypoint].position;
    UpdateLight(patrolLightDistance, patrolLightAngle,    patrolLightIntensity,
    1f);

    if (audioSource.clip != patrolAudio || !audioSource.isPlaying)
    {
          audioSource.clip = patrolAudio;
          audioSource.Play();
    }

    if (agent.remainingDistance < waypointThreshold)
    {
          targetWaypoint = (targetWaypoint + 1) % waypoints.Length;
    }
}
```

Metoda patrol skrbi za robotovo obnašanje, ko se premika med naključnimi točkami. Nastavi primerno hitrost, spremeni zvok, ki ga oddaja ter spremeni velikost svetlobnega žarka in skrbi za izbiro naslednje točke, do katere se bo premaknil.

```
void Chase()
{
    agent.speed = chaseSpeed;
    UpdateLight(chaseLightDistance, chaseLightAngle, ChaseLightIntensity, 1f);
    agent.destination = lastPosition;

    if (audioSource.clip != chaseAudio || !audioSource.isPlaying)
    {
          audioSource.clip = chaseAudio;
          audioSource.Play();
    }


    if (!agent.pathPending && agent.remainingDistance <= agent.stoppingDistance)
    {
          if (!isPlayerDetected)
          {
                isChasing = false;
                Scout();
          }
    }
}
```

Metoda Chase, se izvede ko robot igralca zazna s pomočjo metode Detect, ki jo bom opisal naslednjo. Enako kot v metodi Patrol, se spremeni robotova hitrost in zvok ki ga oddaja.

```
void Detect()
    {
        currentLightDistance = isChasing ? chaseLightDistance : patrolLightDistance;
       currentLightAngle = isChasing ? chaseLightAngle : patrolLightAngle;

       Transform lightOrigin = spotLight.transform;
        isPlayerDetected = false;

       Collider[] detectedColliders = Physics.OverlapSphere(lightOrigin.position,
       currentLightDistance, playerLayer);

        foreach (var collider in detectedColliders)
        {
              Vector3 directionToPlayer = (collider.transform.position -
              lightOrigin.position).normalized;
             float angleToPlayer = Vector3.Angle(lightOrigin.forward,
             directionToPlayer);

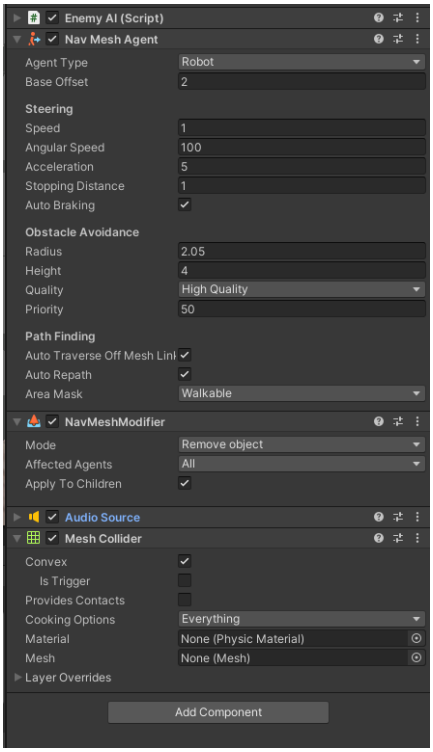
            if (angleToPlayer < currentLightAngle / 2)
            {
                    if (Physics.Raycast(lightOrigin.position, directionToPlayer,
                    out RaycastHit hit, currentLightDistance))
                    {
                           if (hit.collider.gameObject == collider.gameObject)
                    {
                                  isPlayerDetected = true;
                                  break;
                    }
                    }
            } } }
```

Metoda Detect skrbi za zaznavanje igralca. Stalno iz svojega središča pošilja žarke (Raycast) iz leve proti desni. Če žarki zadanejo igralca (torej zadanejo objekt ki se nahaja na plasti »player«, potem se spremenljivka isPlayerDetected nastavi na true, kar v metodi Update zamenja fazo robota na lovljenje (torej se bo izvajala metoda Chase).

```
    void Update()
    {
        if (isPlayerDetected)
        {
            isChasing = true;
            lastPosition = player.position;
            Chase();
        }
        else if (isChasing)
        {
              Chase();
        }
        else
        {
              Patrol();
        }

        Detect();
    }
```

Vse glavne metode stanj robota, so ob primernih pogojih klicane iz metode Update. Metoda Chase, je klicana takrat ko metoda Detect zazna igralca, ki je klicana v funkciji update. Če igralec ni zaznan, se izvaja metoda Patrol.



Na robotu imam poleg te skripte še 4 druge komponente.

Komponente NavMesh Agent določa njegove mere, ki naj bodo uporabljene pri generaciji NavMesh površine in druge nastavitve premikanja.

Komponento NavMeshModifier sem uporabil za to, da pri generaciji površine ne upošteva samega sebe.

Komponenta Audio Source je zadolžena za to, da robot dejansko lahko oddaja nek zvok. Dejanska datoteka, ki pa je uporabljena kot vir zvoka pa se v glavni skripti robota menja ob prehodih stanja.

Komponenta Mesh Collider omogoča robotu, da trka v druge predmete s Collider komponento in ne da gre samo skoznje. Glavna lastnost mesh colliderja je da se sam generira glede na dan 3D model.

4.4 Močne in šibke luči

Glavna funkcionalnost obeh vrst luči, je zaznati, kdaj igralca obsveti. To sem dosegel tako, da sem okoli prostora, kjer svetloba pada na tla, dodal collider in mu obkljukal vrednost isTrigger. Tako lahko v skriptah dostopamo do predmetov, ki so znotraj tega colliderja, kar pomeni da lahko ugotovimo, kdaj je igralec »obsvetljen« z lučjo. Glavni metodi sta OnTriggerEnter in onTriggerExit, ki sta klicani ko objekt pride v trigger collider oziroma ga zapusti. Da pa ima igralec vseeno možnost priti skozi prehode, ki mu jih zapirajo te luči, sem dodal možnost, da lahko luči utripajo.

Skripta WeakLight (šibke luči) in skripta StrongLight (močne luči) dedujeta iz skripte StaticLight, kjer sem jima določil osnovne metode, ki jih potrebujeta obe, znotraj njiju pa sem definiral posebno vedenje.

Močne luči

Ko igralec stopi v šop močne svetlobe, se igralcu prikaže zaslon za konec igre. To sem dosegel s skripto StrongLight.

```
public class StrongLight : StaticLight
{
    protected void Update()
    {
        DetectPlayer();
    }

    protected override void DetectPlayer()
    {
        if (isPlayerInZone && isLightOn)
        {
            ShowDeathScreen();
        }
    }

    private void ShowDeathScreen()
    {
        deathScreen.enabled = true;

        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
    }
}
```

Metode skripte StrongLight delujejo skupaj s dedovanimi skriptami razreda StaticLight. Dodani sta metodi DetectPlayer (pregleda če je igralec v colliderju in če je luč prižgana) in ShowDeathScreen (igralcu omogoči uporabo kurzorja in mu prikaže zaslon ki mu pove da je izgubil). Metoda DetectPlayer se znotraj metode Update, kliče vsako sličico igre.

Šibke luči

Ko igralec stopi v šibek šop svetlobe, robot preide v stanje lovljenja in je njegova ciljna lokacija tam kjer se nahaja igralec ob času zaznave. To sem dosegel s skripto WeakLight

```
public class WeakLight : StaticLight
{

    private void Update()
    {
        DetectPlayer();
    }

    protected override void DetectPlayer()
    {
        if (isPlayerInZone && isLightOn)
        {
            NotifyRobot();
        }
    }

    private void NotifyRobot()
    {
        enemyAi.isPlayerDetected = true;
        enemyAi.lastPosition = detectionCollider.transform.position;
    }       }
```

Metode skripte WeakLight delujejo skupaj s dedovanimi skriptami razreda StaticLight. Dodani sta metodi DetectPlayer (pregleda če je igralec v colliderju in če je luč prižgana) in metoda NotifyRobot (klicana znotraj metode DetectPlayer ko je pogoj pravilen), ki robota prestavi v stanje lovljenja in mu pove lokacijo igralca.

5. Zaključek

V projektu sem skozi strukturirane faze razvil lastno računalniško igro, kar mi je omogočilo, da sem pridobil celovit vpogled v potek izdelave izdelka – od začetne ideje, prototipa in do končne oblike. V uvodnem delu sem oblikoval koncept igre, nato pa sem skozi faze načrtovanja in izdelave postopoma sestavil temeljne mehanike, izgled igralnega prostora ter sistem interakcij, svetlobe in umetne inteligence nasprotnika.

Pri delu sem se srečeval z različnimi izzivi:

- prvič sem se srečal z objektnim programiranjem, vendar sem kar hitro ugotovil potek,
- izgubil sem večino projekta, kar me je pripeljalo do uporabe orodja Git,
- velika količina tehničnih napak

Čeprav projekt v nekaterih delih ni bil izpeljan povsem po začetnih načrtih, sem s končnim rezultatom zadovoljen, saj sem uspel ustvariti delujočo igro z značilnim vzdušjem.

6. Literatura in viri

Brackeys. 2020. FPS Controller [YouTube video]. Dostopno prek: https://www.youtube.com/watch?v=_QajrabyTJc& (10. 12. 2024).

Rehope Games. 2023. Adding music and sound effects to a game [YouTube video]. Dostopno prek: https://www.youtube.com/watch?v=N8whM1GjH4w (15. 3. 2025).

Rehope Games. 2023. Unity audio volume [YouTube video]. Dostopno prek: https://www.youtube.com/watch?v=G-JUp8AMEx0& (25. 3. 2025).

Rehope Games. 2024. Making a main menu [YouTube video]. Dostopno prek: https://www.youtube.com/watch?v=DX7HyN7oJjE (18. 3. 2025).

Meshy AI. Dostopno prek: https://www.meshy.ai/workspace (20. 2. 2025).

Code Monkey. 2022. How to use unity NavMesh [YouTube video]. Dostopno prek: https://www.youtube.com/watch?v=atCOd4o7tG4 (10. 1. 2024).

Unity Asset Store. 2022. Locker Package. Dostopno prek: https://assetstore.unity.com/packages/3d/props/locker-56405 (15. 2. 2025).

Unity Asset Store. 2022. Key and Lock Furniture Package. Dostopno prek: https://assetstore.unity.com/packages/3d/props/furniture/key-and-lock-193317 (25. 2. 2025).

Unity Asset Store. 2016. Office Room Furniture Package. Dostopno prek: https://assetstore.unity.com/packages/3d/props/furniture/office-room-furniture-70884 (28. 12. 2024).

Unity Technologies. 2023. Unity Script Reference. Dostopno prek: https://docs.unity3d.com/ScriptReference/ (20. 3. 2025).

Unity Asset Store. 2016. Bodyguards Humanoid Package. Dostopno prek: https://assetstore.unity.com/packages/3d/characters/humanoids/humans/bodyguards-31711 (5. 3. 2025).

Unity Asset Store. 2019. Electrical Shield. Dostopno prek: https://assetstore.unity.com/packages/3d/props/electrical-shield-118266 (5. 3. 2025).

Unity Asset Store. 2024. Surveillance Camera Prop. Dostopno prek: https://assetstore.unity.com/packages/3d/props/surveillance-camera-264577 (22. 12. 2024).

Unity Asset Store. 2024. Keypad Free Package. Dostopno prek: https://assetstore.unity.com/packages/3d/props/electronics/keypad-free-262151 (30. 2. 2025).

Unity Asset Store. 2022. PBR Metal Barrels Pack. Dostopno prek: https://assetstore.unity.com/packages/3d/props/industrial/pbr-metal-barrels-pack-221309 (15. 1. 2025).

Unity Asset Store. Oil Cans Prop. Dostopno prek: https://assetstore.unity.com/packages/3d/props/oil-cans-841 (15. 12. 2024). - produkt ne obstaja več

Unity Asset Store. Tinned Food Package. Dostopno prek: https://assetstore.unity.com/packages/3d/props/food/tinned-food-89246 (15. 12. 2024). - produkt ne obstaja več

Unity Asset Store. 2016. Forklift Prop. Dostopno prek: https://assetstore.unity.com/packages/3d/props/industrial/forklift-60960 (4. 1. 2025).

Unity Asset Store. 2018. Low Poly Storage Pack. Dostopno prek: https://assetstore.unity.com/packages/3d/environments/urban/low-poly-storage-pack-101732 (10. 12. 2024).

Unity Asset Store. 2024. Wood Box Pack – 15 Objects. Dostopno prek: https://assetstore.unity.com/packages/3d/props/industrial/wood-box-pack-15-objects-105811 (15. 12. 2024).

Unity Asset Store. 2018. One-Click Wall Materials. Dostopno prek: https://assetstore.unity.com/packages/2d/textures-materials/one-click-wall-materials-117123 (6. 3. 2025).

OpenAI. 2022. ChatGPT Online. Dostopno prek: https://chatgpt.com/ (31. 3. 2025)

High-Flyer. 2023. Deepseek. Dostopno prek: https://chat.deepseek.com/ (17. 3. 2025)

Izjava o avtorstvu

Izjavljam, da je strokovno poročilo Prvoosebna 3D računalniška igra v pogonu Unity v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

16. 4. 2025 Boštjan Skubic

